

Programmentwurf

[Bezeichnung]

Name: [Name, Vorname]

Matrikelnummer: [MNR]

Abgabedatum: [DATUM]

Allgemeine Anmerkungen:

- *es darf nicht auf andere Kapitel als Leistungsnachweis verwiesen werden (z.B. in der Form “XY wurde schon in Kapitel 2 behandelt, daher hier keine Ausführung”)*
- *alles muss in UTF-8 codiert sein (Text und Code)*
- *sollten mündliche Aussagen den schriftlichen Aufgaben widersprechen, gelten die schriftlichen Aufgaben (ggf. an Anpassung der schriftlichen Aufgaben erinnern!)*
- *alles muss ins Repository (Code, Ausarbeitung und alles was damit zusammenhängt)*
- *die Beispiele sollten wenn möglich vom aktuellen Stand genommen werden*
 - *finden sich dort keine entsprechenden Beispiele, dürfen auch ältere Commits unter Verweis auf den Commit verwendet werden*
 - *Ausnahme: beim Kapitel “Refactoring” darf von vorne herein aus allen Ständen frei gewählt werden (mit Verweis auf den entsprechenden Commit)*
- *falls verlangte Negativ-Beispiele nicht vorhanden sind, müssen entsprechend mehr Positiv-Beispiele gebracht werden*
 - *Achtung: werden im Code entsprechende Negativ-Beispiele gefunden, gibt es keine Punkte für die zusätzlichen Positiv-Beispiele*
 - *Beispiele*
 - *“Nennen Sie jeweils eine Klasse, die das SRP einhält bzw. verletzt.”*
 - *Antwort: Es gibt keine Klasse, die SRP verletzt, daher hier 2 Klassen, die SRP einhalten: [Klasse 1], [Klasse 2]*
 - *Bewertung: falls im Code tatsächlich keine Klasse das SRP verletzt: volle Punktzahl ODER falls im Code mind. eine Klasse SRP verletzt: halbe Punktzahl*
- *verlangte Positiv-Beispiele müssen gebracht werden*
- *Code-Beispiel = Code in das Dokument kopieren*

Kapitel 1: Einführung

Übersicht über die Applikation

[Was macht die Applikation? Wie funktioniert sie? Welches Problem löst sie/welchen Zweck hat sie?]

Wir machen ein Spiel namens „Catch That Cow!“, in dem man Tiere fängt. Das Interface des Spiels ist Textbasiert. Die Tiere leben in verschiedenen Biomen, durch die man sich bewegen kann. Tiere haben Stats in verschiedenen Kategorien (z.B. Geschwindigkeit, Länge, Gewicht, ...). Die Stats hängen von ihrer Tierart ab und variieren zufällig leicht.

Tiere fängt man, indem man sie im Kampf besiegt. Ein Kampf funktioniert so:

- Man begegnet dem Tier „Eine wilde Kobra erscheint!“
- Handelt es sich um ein Raubtier, startet das Tier in der ersten Runde mit dem Angriff, bei Fluchttieren kann der Spieler zuerst angreifen oder abhauen. Anschliessend werden die Runden abwechselnd begonnen.
- Der Spieler hat ein Kampfinventar von 3 aus seinem Inventar zufällig ausgewählten Tieren mit denen er gegen das Gegnertier kämpfen kann.
- Der Kampf geht so lange, bis entweder der Spieler oder das Gegnertier 3 Punkte hat
- Die Runde beginnt mit einem Angriff in einer bestimmten Kategorie. Wenn das Gegnertier angreift, wird die Kategorie zufällig ausgewählt. Greift der Spieler an, wählt er ein Tier aus seinem Kampfinventar und eine Kategorie aus. Die Runde gewinnt, wer die besseren Stats in der gewählten Kategorie hat. Der Gewinner der Runde erhält einen Punkt.
- Das Tier aus dem Kampfinventar, das der Spieler in dieser Runde ausgewählt hat, wird für weitere Runden deaktiviert.

Verliert der Spieler den Kampf, so wird zufällig eines der Tiere im Kampfinventar gelöscht. Gewinnt der Spieler den Kampf, so erhält er das Gegnertier in sein Inventar. Nach dem Kampf kommen alle Tiere aus dem Kampfinventar wieder ins Inventar.

Man kann in einem Inventar sehen, welche Tiere man bereits gefangen hat und welche Stats diese haben.

Der Spieler startet in einem Startbiom. Es gibt eine Karte aus $(2n+1)^2$ Biomen. In jedem Biom kann der Spieler auswählen, in welche Himmelsrichtung er weitergehen möchte, und welche Biome in dort erwarten (z.B. Norden: Wüste, Osten: Tundra, Süden: Jungle, ...) oder ob er in dem Biom bleiben möchte.

Die Tierarten sind in verschiedene Level eingeteilt (z.B. Biene ist level 1, Eisbaer level 3) Biome mit Distanz 1 zum Startbiom enthalten Tiere mit Level 1, usw.

Wie startet man die Applikation?

[Wie startet man die Applikation? Welche Voraussetzungen werden benötigt? Schritt-für-Schritt-Anleitung]

Wie testet man die Applikation?

[Wie testet man die Applikation? Welche Voraussetzungen werden benötigt? Schritt-für-Schritt-Anleitung]

Kapitel 2: Clean Architecture

Was ist Clean Architecture?

[allgemeine Beschreibung der Clean Architecture in eigenen Worten]

Analyse der Dependency Rule

[(1 Klasse, die die Dependency Rule einhält und eine Klasse, die die Dependency Rule verletzt); jeweils UML der Klasse und Analyse der Abhängigkeiten in beide Richtungen (d.h., von wem hängt die Klasse ab und wer hängt von der Klasse ab) in Bezug auf die Dependency Rule]

Positiv-Beispiel: Dependency Rule

Negativ-Beispiel: Dependency Rule

Analyse der Schichten

[jeweils 1 Klasse zu 2 unterschiedlichen Schichten der Clean-Architecture: jeweils UML der Klasse (ggf. auch zusammenspielenden Klassen), Beschreibung der Aufgabe, Einordnung mit Begründung in die Clean-Architecture]

Schicht: [Name]

Schicht: [Name]

Kapitel 3: SOLID

Analyse Single-Responsibility-Principle (SRP)

[jeweils eine Klasse als positives und negatives Beispiel für SRP; jeweils UML der Klasse und Beschreibung der Aufgabe bzw. der Aufgaben und möglicher Lösungsweg des Negativ-Beispiels (inkl. UML)]

Positiv-Beispiel

Negativ-Beispiel

Analyse Open-Closed-Principle (OCP)

[jeweils eine Klasse als positives und negatives Beispiel für OCP; jeweils UML der Klasse und Analyse mit Begründung, warum das OCP erfüllt/nicht erfüllt wurde – falls erfüllt: warum hier sinnvoll/welches Problem gab es? Falls nicht erfüllt: wie könnte man es lösen (inkl. UML)?]

Positiv-Beispiel

Negativ-Beispiel

Analyse Liskov-Substitution- (LSP), Interface-Segregation- (ISP), Dependency-Inversion-Principle (DIP)

[jeweils eine Klasse als positives und negatives Beispiel für entweder LSP oder ISP oder DIP); jeweils UML der Klasse und Begründung, warum man hier das Prinzip erfüllt/nicht erfüllt wird]

[Anm.: es darf nur ein Prinzip ausgewählt werden; es darf NICHT z.B. ein positives Beispiel für LSP und ein negatives Beispiel für ISP genommen werden]

Positiv-Beispiel

Negativ-Beispiel

Kapitel 4: Weitere Prinzipien

Analyse GRASP: Geringe Kopplung

[jeweils eine bis jetzt noch nicht behandelte Klasse als positives und negatives Beispiel geringer Kopplung; jeweils UML Diagramm mit zusammenspielenden Klassen, Aufgabenbeschreibung und Begründung für die Umsetzung der geringen Kopplung bzw. Beschreibung, wie die Kopplung aufgelöst werden kann]

Positiv-Beispiel

Negativ-Beispiel

Analyse GRASP: Hohe Kohäsion

[eine Klasse als positives Beispiel hoher Kohäsion; UML Diagramm und Begründung, warum die Kohäsion hoch ist]

Don't Repeat Yourself (DRY)

[ein Commit angeben, bei dem duplizierter Code/duplizierte Logik aufgelöst wurde; Code-Beispiele (vorher/nachher); begründen und Auswirkung beschreiben]

Kapitel 5: Unit Tests

10 Unit Tests

[Nennung von 10 Unit-Tests und Beschreibung, was getestet wird]

Unit Test	Beschreibung
<i>Klasse#Methode</i>	

ATRIP: Automatic

[Begründung/Erläuterung, wie 'Automatic' realisiert wurde]

ATRIP: Thorough

[jeweils 1 positives und negatives Beispiel zu 'Thorough'; jeweils Code-Beispiel, Analyse und Begründung, was professionell/nicht professionell ist]

ATRIP: Professional

[jeweils 1 positives und negatives Beispiel zu 'Professional'; jeweils Code-Beispiel, Analyse und Begründung, was professionell/nicht professionell ist]

Code Coverage

[Code Coverage im Projekt analysieren und begründen]

Fakes und Mocks

[Analyse und Begründung des Einsatzes von 2 Fake/Mock-Objekten; zusätzlich jeweils UML Diagramm der Klasse]

Kapitel 6: Domain Driven Design

Ubiquitous Language

[4 Beispiele für die Ubiquitous Language; jeweils Bezeichnung, Bedeutung und kurze Begründung, warum es zur Ubiquitous Language gehört]

Bezeichnung	Bedeutung	Begründung
-------------	-----------	------------

Entities

[UML, Beschreibung und Begründung des Einsatzes einer Entity; falls keine Entity vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]

Value Objects

[UML, Beschreibung und Begründung des Einsatzes eines Value Objects; falls kein Value Object vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]

Repositories

[UML, Beschreibung und Begründung des Einsatzes eines Repositories; falls kein Repository vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]

Aggregates

[UML, Beschreibung und Begründung des Einsatzes eines Aggregates; falls kein Aggregate vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]

Kapitel 7: Refactoring

Code Smells

[jeweils 1 Code-Beispiel zu 2 Code Smells aus der Vorlesung; jeweils Code-Beispiel und einen möglichen Lösungsweg bzw. den genommen Lösungsweg beschreiben (inkl. (Pseudo-)Code)]

2 Refactorings

[2 unterschiedliche Refactorings aus der Vorlesung anwenden, begründen, sowie UML vorher/nachher liefern; jeweils auf die Commits verweisen]

Kapitel 8: Entwurfsmuster

[2 unterschiedliche Entwurfsmuster aus der Vorlesung (oder nach Absprache auch andere) jeweils sinnvoll einsetzen, begründen und UML-Diagramm]

Entwurfsmuster: [Name]

Entwurfsmuster: [Name]